CS 554 Project Proposal: Analyzing and Restoring Mutual Exclusion in Lock Algorithms Across Hardware and Language Memory Models

Krishna Riteshkumar Patel (UIN: 670178497)
CS 554, Fall 2026

Due: October 23, 2026

1 Project Overview and Goals

In class, we studied the SC-DRF language memory model with locks by treating Acq ℓ and Rel ℓ as built-in primitives that establish sw (synchronizes-with) edges whenever a release precedes an acquire in the lock order lo. In practice, however, lock primitives do not exist in a vacuum: systems programmers and standard library authors must implement them using lower-level shared-memory reads, writes, fences, and atomic read-modify-write (RMW) instructions running over weak memory models.

The goal of this project is to formally analyze classic mutual exclusion and synchronization algorithms—specifically **Peterson's Lock**, **Dekker's Lock**, **Ticket Spinlocks**, and a simplified **Read-Copy-Update (RCU)** quiescent-state protocol—across both axiomatic hardware memory models (SC, TSO, RA, and SRA) and the C/C++11 language memory model. Specifically, I plan to:

1. Encode bounded unrollings of these lock acquisition/release protocols as litmus tests and verify whether mutual exclusion (at most one thread in the critical section) holds under SC, TSO, RA, and SRA.

2. Extract concrete counterexample execution graphs demonstrating how weak memory behaviors (such as write-to-read reordering in TSO and per-location coherence in RA) allow both threads to enter the critical section simultaneously.

3. Evaluate and compare different synchronization mechanisms to restore correctness, including standard hardware fences (fence in TSO) and Lahav et al.'s encoding of SC fences as release-acquire RMW updates ($\langle f := 0 \rangle$) to a dummy location under RA and SRA.

4. Implement these algorithms in C11 using `stdatomic.h` and use an automated stateless model checker to test how `memory_order_relaxed`, `memory_order_acq_rel`, and `memory_order_seq_cst` affect correctness and state-space exploration.

2 Models and Tools

- **Axiomatic Memory Models:** SC, TSO, RA, and SRA (extended with fence and atomic update/RMW events), alongside the C11 memory model.

- **Herd7 (`herdtools7`):** I will write custom `.cat` files for our course definitions of SC, TSO, RA, and SRA, and run `herd7` on custom `.litmus` files representing lock entry/exit critical sections to automatically check whether a mutual exclusion violation state is reachable.

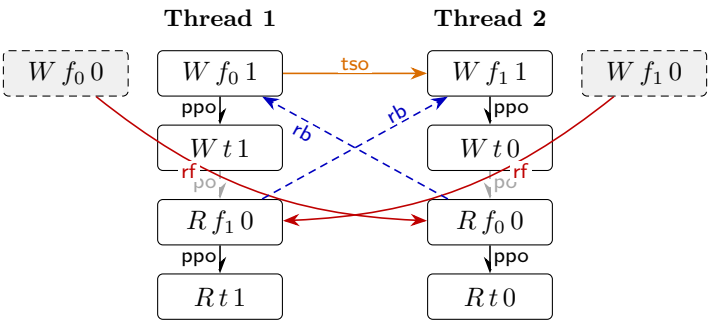
- **CDSChecker:** To explore a verification tool not covered in class, I will use **CDSChecker** (an exhaustive stateless model checker for C/C++11 concurrent programs). Unlike `herd7`, which checks fixed assembly-style litmus tests, CDSChecker executes actual C11 code with loops and dynamic control flow, allowing me to test full lock acquisitions and detect data races or assertion failures inside critical sections.

3 Concrete Example of Expected Results

Consider a two-thread litmus test for the entry protocol of **Peterson's Lock**. Shared variables `flag0`, `flag1`, and `turn` are initially 0. Thread 1 sets `flag0` = 1 and `turn` = 1, then reads `flag1` into `r1` and `turn` into `r2`; it enters the critical section if `r1` = 0 or `r2` = 0. Symmetrically, Thread 2 sets `flag1` = 1 and `turn` = 0, then reads `flag0` into `r3` and `turn` into `r4`; it enters the critical section if `r3` = 0 or `r4` = 1.

Thread 1	Thread 2
`store flag0 1`	`store flag1 1`
`store turn 1`	`store turn 0`
`r1 = load flag1`	`r3 = load flag0`
`r2 = load turn`	`r4 = load turn`

Forbidden Outcome (Mutual Exclusion Violation):
`r1 = 0` and `r3 = 0` (both threads see the other's flag as 0 and enter the critical section at the same time).



Under SC, the outcome `r1 = 0, r3 = 0` is impossible because the write-to-read `po` edges force a cycle with the `rb` edges. Under TSO (shown above, with f_0, f_1, t abbreviating `flag0`, `flag1`, `turn`), the edges $W\,t\,1 \xrightarrow{\text{po}} R\,f_1\,0$ and $W\,t\,0 \xrightarrow{\text{po}} R\,f_0\,0$ are write-to-read edges and are excluded from `ppo`. Consequently, $\mathsf{hb} := \mathsf{ppo} \cup \mathsf{tso} \cup \mathsf{rf} \cup \mathsf{rb}$ is acyclic (every `hb` edge points forward in the order $W\,f_0\,0 \to W\,f_1\,0 \to R\,f_1\,0 \to R\,f_0\,0 \to W\,f_0\,1 \to W\,t\,1 \to R\,t\,1 \to W\,f_1\,1 \to W\,t\,0 \to R\,t\,0$). Both threads can therefore enter the critical section simultaneously, breaking mutual exclusion! Adding a fence between the store to `turn` and the load of the other thread's flag restores the `ppo` edge and forbids the bad execution.

4 Motivation and Stretch Goals

I am interested in this project because it connects the theoretical axiomatic models we studied in Homeworks 2 and 3 directly to practical concurrent programming in C/C++. While it is easy to assume that standard lock algorithms from operating systems textbooks work out of the box, analyzing them in `herd7` and CDSChecker shows why modern lock implementations rely heavily on specific memory orders and atomic RMWs. If time permits, I will also model a lock-free data structure (such as a single-producer/single-consumer ring buffer) in CDSChecker and prove that replacing `memory_order_seq_cst` with release-acquire atomics (RA/SRA) preserves correctness without requiring full fences.